

Topic 7: Unsupervised Learning — Clustering

Instructor: Dr. Akash Gupta

Table of contents

1	What Is Clustering?	2
2	The Building Block: A Single Cluster	2
3	K-Means: The Simplest Algorithm	3
3.1	How K-Means Works	3
3.2	A Worked Example	4
3.3	Building K-Means in Python	4
4	How Do You Pick K?	5
4.1	Method 1: The Elbow Method	5
4.2	Method 2: The Silhouette Score	6
4.3	Combining Both: Best Practice	6
5	Hierarchical Clustering: When K Is Not Known in Advance	8
5.1	How It Works (Bottom-Up Version)	8
5.2	Reading a Dendrogram: A Worked Example	8
5.3	How “Closest” Is Measured: Linkage	10
5.4	Pros and Cons	10
6	Comparing the Two Algorithms	11
7	Putting It Together: A Customer Segmentation Workflow	11
8	Summary	13

1 What Is Clustering?

Most predictive models studied so far require **labeled data** — the model is provided with the correct answer for every training example and learns to predict that answer on new data. Clustering is fundamentally different. **Clustering examines unlabeled data and seeks to discover natural groupings on its own.**

Given a collection of customers, songs, or transactions, clustering identifies which records belong together without any pre-defined labels.

Real businesses use clustering everywhere:

- **Sephora** segments its 30 million Beauty Insider members into groups for personalized marketing campaigns
- **Spotify** clusters users by listening behavior to power Discover Weekly and Daily Mixes
- **Netflix** groups subscribers with similar viewing patterns to test new recommendation algorithms
- **Stitch Fix** clusters customers by style profile so stylists can pull similar items
- **Starbucks** clusters store locations by foot traffic patterns to plan new store openings
- **Amazon** clusters products by purchase patterns (“customers who bought this also bought...”)

In every case, no labels were provided in advance; the algorithm identified the groupings from the data alone.

Key Takeaway — Clustering Is Descriptive, Not Predictive

Unlike a churn model that produces a single “right” answer, clustering has no ground truth. Different algorithms and different parameter choices will yield different valid groupings of the same data. The role of the analyst is to select a clustering that is **useful for the business question at hand**, not to identify the “correct” one.

2 The Building Block: A Single Cluster

Before introducing any specific algorithm, it is useful to define the smallest unit in clustering: the **cluster** itself.

- A **cluster** is a group of data points that are more similar to each other than they are to points outside the group
- Similarity is typically measured by **distance** — points that lie close together in feature space are considered similar
- For Sephora customers, “close” may indicate similar age, similar spend, and similar product categories
- For Spotify songs, “close” may indicate similar tempo, similar energy, and similar genre tags
- A well-formed cluster is **tight** (its members lie close to one another) and **separated** (its members lie far from members of other clusters)

This is the central principle of clustering. Every clustering algorithm is simply a different procedure for identifying groups that are tight and well-separated.

Key Takeaway — Always Scale Your Features First

Clustering relies on distances. If one feature is measured in dollars (range \$0–\$10,000) and another in years (range 18–80), the dollar feature will dominate the distance calculation simply because its numerical values are larger. **Always standardize features before clustering** so that each feature contributes proportionally.

3 K-Means: The Simplest Algorithm

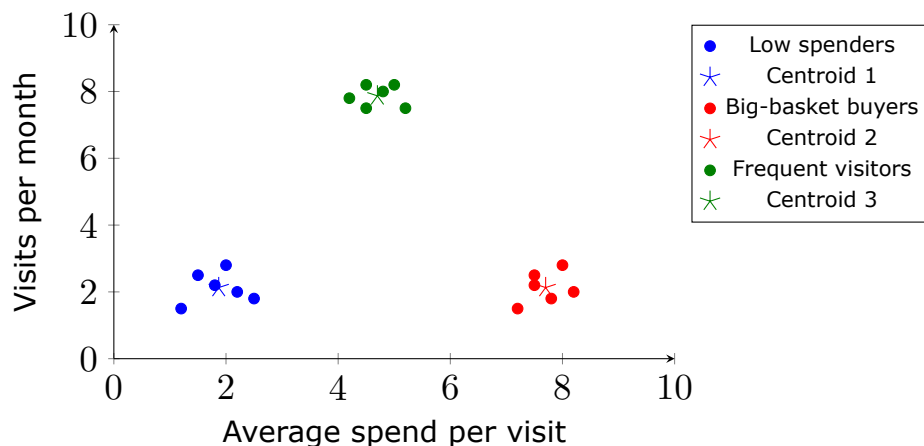
The most widely used clustering algorithm is **K-Means**. The letter K denotes the number of clusters, which the analyst specifies in advance. K-Means is popular because it is fast, intuitive, and effective for most business problems.

3.1 How K-Means Works

Suppose the goal is to organize a Sephora customer database into 4 groups. K-Means proceeds as follows:

- **Step 1 — Initialize centroids.** Place 4 starting points (called **centroids**) at random locations within the customer feature space.
- **Step 2 — Assign every customer.** For each customer, identify the nearest centroid and assign the customer to that cluster.
- **Step 3 — Update each centroid.** Recompute each centroid as the average position of the customers assigned to it.
- **Step 4 — Repeat.** Reassign customers based on the updated centroid positions, then update the centroids again. Continue until the centroids no longer move.

This iterative process defines the entire algorithm. The resulting clusters consist of customers who are each closest to the same centroid.



3.2 A Worked Example

To illustrate the algorithm concretely, consider K-Means applied to three customers using a single feature (annual spend in thousands of dollars): \$1K, \$2K, and \$10K. Set $K = 2$ clusters.

- **Initialize:** place two centroids — Centroid A at \$1, Centroid B at \$10.
- **Iteration 1, assignment step:**
 - Customer at \$1 is closer to A → assigned to cluster A
 - Customer at \$2 is closer to A (distance 1) than to B (distance 8) → assigned to cluster A
 - Customer at \$10 is closer to B → assigned to cluster B
- **Iteration 1, update step:**
 - Centroid A moves to the average of \$1 and \$2, which is **\$1.5**
 - Centroid B remains at **\$10** (only one customer is assigned to it)
- **Iteration 2, assignment step:** No customer changes cluster. The algorithm has **converged**.

Final result: cluster A contains {\$1, \$2} (low spenders); cluster B contains {\$10} (high spender). A realistic Sephora analysis would involve thousands of customers and many features, but the underlying procedure is identical — only the scale differs.

3.3 Building K-Means in Python

In practice, K-Means is never computed by hand. scikit-learn provides a concise implementation:

```
from sklearn.cluster import KMeans

kmeans = KMeans(n_clusters=4, random_state=42, n_init=10)
kmeans.fit(X_scaled)

labels = kmeans.labels_          # which cluster each row belongs to
centers = kmeans.cluster_centers_ # the coordinates of each centroid
```

Try It in the Notebook

The companion notebook walks through K-Means on a real customer dataset — scaling features, running the algorithm, profiling each cluster to characterize its members, and assigning business-friendly names to each segment. Complete the hands-on exercises there.

Under the Hood — Optional Formula

For readers who wish to see the formal definition: K-Means minimizes the **within-cluster sum of squares (WCSS)**, which is the total squared distance from every point to its assigned centroid:

$$\text{WCSS} = \sum_{k=1}^K \sum_{x_i \in C_k} \|x_i - \mu_k\|^2$$

In plain language: sum the squared distance from every point to its assigned centroid across all clusters. Smaller WCSS indicates tighter, better-separated clusters. scikit-learn reports this value as `kmeans.inertia_`; it is not computed by hand.

4 How Do You Pick K?

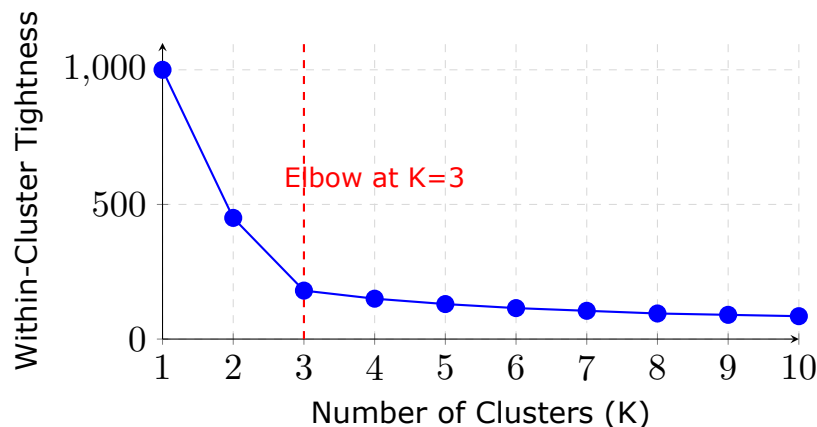
The principal practical challenge in K-Means is deciding how many clusters to specify. Choosing too few merges distinct groups together; choosing too many fragments meaningful groups into uninformative sub-segments. Two standard methods help guide this choice.

4.1 Method 1: The Elbow Method

The elbow method is a visual heuristic:

- Run K-Means for several values of K (say 1, 2, 3, ..., 10)
- For each K, record how tight the clusters are (the within-cluster sum of squares — lower is better)
- Plot tightness on the y-axis vs. K on the x-axis
- Look for the “**elbow**” — the point where the curve bends sharply, after which adding more clusters barely helps

The intuition: every additional cluster makes the clusters slightly tighter, but the marginal gains diminish past a certain point. That bend identifies an appropriate value of K.



In this example, the curve bends sharply at $K=3$. Beyond that point, additional clusters yield negligible improvement. $K=3$ is the elbow.

4.2 Method 2: The Silhouette Score

The silhouette score is a number between -1 and $+1$ that says how well-clustered each point is:

- **Close to +1** — the point is much closer to its own cluster than to any other. Well clustered.
- **Close to 0** — the point lies on the boundary between two clusters. Cluster assignment is ambiguous.
- **Negative** — the point is closer to a different cluster than the one to which it was assigned. Likely misclassified.

The average silhouette is computed across all points. Values above 0.5 generally indicate strong cluster structure; values below 0.25 indicate that the clusters overlap substantially.

```
from sklearn.metrics import silhouette_score

score = silhouette_score(X_scaled, kmeans.labels_)
print(f"Silhouette score: {score:.3f}")
```

Under the Hood — Optional Silhouette Formula

For readers who wish to see the formal definition: for each point, the silhouette compares two distances — a (the average distance to other points in the same cluster) and b (the average distance to points in the nearest other cluster):

$$s = \frac{b - a}{\max(a, b)}$$

In plain language: how much closer is the point to its own cluster than to its nearest alternative? Higher values indicate better clustering. scikit-learn computes this with `silhouette_score()`.

4.3 Combining Both: Best Practice

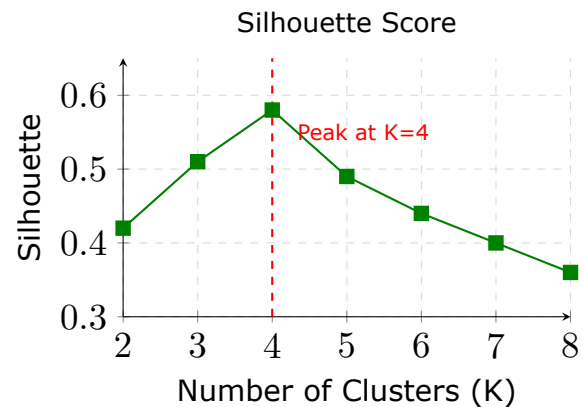
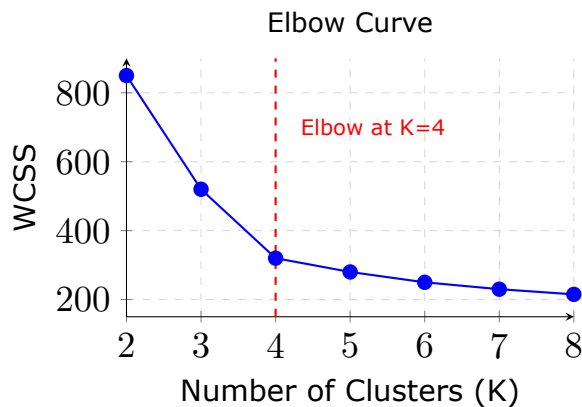
Use **both** methods in combination:

- Run K-Means for $K = 2, 3, \dots, 10$
- Plot the elbow curve
- Plot the silhouette score over the same range
- Select a K that lies near the elbow AND near the silhouette peak

4.3.1 Worked Example: Spotify Listener Segmentation

Suppose Spotify wants to segment 50,000 active listeners using four behavioral features: hours listened per week, genre diversity, share rate, and skip rate. The data science team runs K-Means for $K = 2$ through 8 and records both diagnostics:

K	WCSS (tightness)	Silhouette score
2	850	0.42
3	520	0.51
4	320	0.58
5	280	0.49
6	250	0.44
7	230	0.40
8	215	0.36



Reading the diagnostics:

- **Elbow:** WCSS drops sharply from $K=2$ to $K=4$, then flattens — the elbow is at $K=4$
- **Silhouette:** the score peaks at 0.58 when $K=4$ and declines for larger K — best separation also occurs at $K=4$
- **Both methods agree.** Spotify selects $K=4$ and proceeds to profile the four resulting segments

After profiling, the four segments turn out to be Power Listeners'' (high hours, broad genres), Casual Streamers'' (low hours, narrow taste), Genre Specialists'' (high hours, low diversity), and Discovery Seekers'' (high share rate, high skip rate). All four are nameable and actionable for the marketing team.

When the Two Methods Disagree

The diagnostics will not always agree. If the elbow points to $K=4$ but the silhouette peaks at $K=6$, prefer the value of K that produces segments the business can name and act upon. A slightly lower silhouette score with four interpretable segments is preferable to six segments that no marketer can describe.

Ultimately, the appropriate K depends on **business usefulness**. If K=4 produces four customer segments that the marketing team can act upon, it is preferable to K=7, which may yield a marginally higher score but seven segments that resist clear naming.

5 Hierarchical Clustering: When K Is Not Known in Advance

K-Means requires the analyst to specify K in advance. **Hierarchical clustering** does not — instead, it builds a tree (called a **dendrogram**) that shows how points group together at every possible level. The tree can then be “cut” at any desired number of clusters after the fact.

5.1 How It Works (Bottom-Up Version)

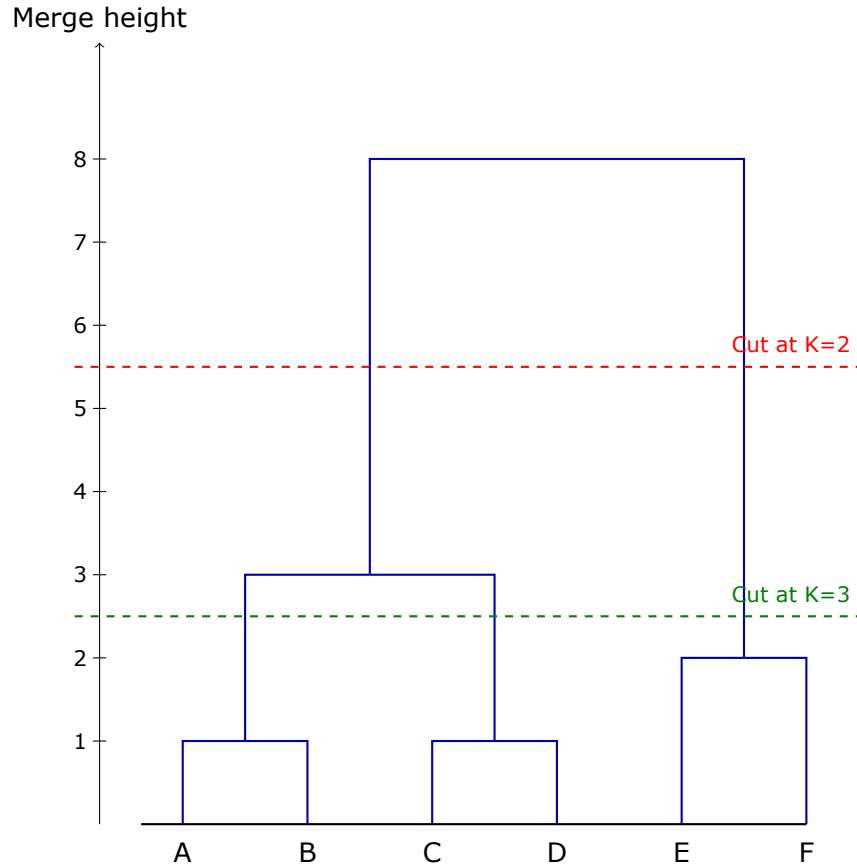
The most common variant is called **agglomerative** clustering. It proceeds bottom-up:

- Begin with every point as its own single-member cluster
- Identify the two closest clusters and merge them into one
- Identify the next two closest clusters and merge them
- Continue merging until all points belong to a single cluster
- The result is a tree that records every merge in sequence

To obtain K clusters, cut the tree at the height that produces exactly K branches.

5.2 Reading a Dendrogram: A Worked Example

Suppose Stitch Fix wants to group six customers based on annual spend (in dollars): A=\$10, B=\$15, C=\$50, D=\$55, E=\$200, F=\$220. Agglomerative clustering with Ward linkage produces the dendrogram below. The horizontal axis lists customers; the vertical axis shows the **merge height** (how dissimilar the two merged clusters were at the moment of merging).



Customer spend levels: A=\$10, B=\$15, C=\$50, D=\$55, E=\$200, F=\$220.

How to read this dendrogram:

- **Each horizontal bar** represents a merge. Its height shows how far apart the two merged clusters were
- **Short bars near the bottom** indicate very similar items merging — A and B (both around \$10–\$15) merged first
- **Tall bars near the top** indicate dissimilar groups merging — the high-spender pair (E, F) merged with the rest only at height 8
- **A large vertical gap** between merge heights signals a natural separation in the data

Now use a horizontal **cut** to choose K:

- **Cut at K=2** (red dashed line near the top): the tree splits into two clusters — {A, B, C, D} and {E, F}, separating low/mid spenders from high spenders
- **Cut at K=3** (green dashed line lower down): the tree splits into three clusters — {A, B}, {C, D}, and {E, F}, distinguishing low, mid, and high spenders

The big jump in merge height between 3 and 8 is a strong signal that K=2 captures the most meaningful split in the data — there is a clear gap between the two groups.

Key Takeaway — Look for the Tallest Vertical Gap

The most natural number of clusters is found by identifying the **tallest vertical gap between consecutive merges** on the dendrogram. Cut horizontally through that gap. In the example above, the gap between height 3 and height 8 is by far the largest, indicating $K=2$ as the most defensible choice.

5.3 How “Closest” Is Measured: Linkage

When two clusters each contain many points, “the distance between them” can be defined in several different ways:

- **Single linkage** — distance between the two closest points (one from each cluster)
- **Complete linkage** — distance between the two farthest points
- **Average linkage** — average of all pairwise distances
- **Ward linkage** — merges the two clusters that increase the within-cluster variance the least

Key Takeaway — Use Ward Linkage by Default

Ward linkage typically produces the most balanced and interpretable clusters and is the modern default in most business applications. Other linkages are useful in specific cases (e.g., single linkage can identify chain-shaped clusters), but Ward is the recommended starting point.

```
from scipy.cluster.hierarchy import linkage, dendrogram
from sklearn.cluster import AgglomerativeClustering

# Build linkage matrix and plot dendrogram
Z = linkage(X_scaled, method="ward")
dendrogram(Z)

# Or cut the tree at K=4 directly
agg = AgglomerativeClustering(n_clusters=4, linkage="ward")
labels = agg.fit_predict(X_scaled)
```

5.4 Pros and Cons**Advantages:**

- K need not be specified in advance — the dendrogram allows the analyst to select it after examining the structure
- The dendrogram reveals nested relationships (e.g., “high spenders subdivide further into urban and suburban high spenders”)
- Deterministic — no random initialization, so the same result is produced every run

Disadvantages:

- Slow on large datasets — becomes impractical beyond tens of thousands of rows
- Once two points are merged, they cannot be separated again — early mistakes propagate
- Less flexible than K-Means at scale

In practice: **hierarchical clustering is suited to small exploratory analyses (a few thousand customers), while K-Means is preferred for production-scale work.**

6 Comparing the Two Algorithms

The following table summarizes how to choose between K-Means and hierarchical clustering:

K-Means	Hierarchical
Pros: Fast, scalable, and intuitive Cons: K must be specified in advance; performs poorly on non-spherical clusters; sensitive to outliers Use when: Clusters are expected to be approximately spherical and similarly sized — the most common business case	Pros: No K required in advance; dendrogram visualization reveals nested structure Cons: Slow on large datasets; merges are irreversible Use when: The dataset is small (a few thousand rows) and visual exploration of cluster structure is desired

7 Putting It Together: A Customer Segmentation Workflow

The most common business application of clustering is **customer segmentation** for personalized marketing. The typical end-to-end workflow proceeds as follows:

- **Step 1 — Select features that describe behavior.** For Sephora: average spend, visit frequency, recency of last purchase, product categories purchased, and response to promotions
- **Step 2 — Clean and scale the data.** Handle missing values, then standardize every feature so that distance comparisons are not dominated by units
- **Step 3 — Run K-Means at several values of K.** Fit the model for $K = 2, 3, \dots, 10$ and record both the elbow tightness and the silhouette score
- **Step 4 — Select K.** Use the elbow and silhouette plots in combination with the business question: "How many segments can the marketing team operationalize?" Most organizations select between 3 and 8
- **Step 5 — Profile each cluster.** Compute the mean of each feature within each cluster to characterize the members — high spenders, lapsed customers, weekend visitors, and so on
- **Step 6 — Assign each segment a business-friendly name.** Examples: Power Users'', At-Risk Loyalists'', Bargain Hunters'', New Trials''. Clear naming is essential, as these labels become the language of day-to-day marketing operations
- **Step 7 — Hand off to marketing or product teams.** Each segment receives tailored campaigns, product recommendations, and retention strategies

This is the workflow that Sephora, Stitch Fix, and Spotify use to construct the segments underlying their personalization engines.

Practice — K-Means by Hand on Annual Spend

A small Etsy seller has six customers with annual spend (in dollars): \$2, \$4, \$6, \$14, \$16, \$18. Run K-Means with $K = 2$ and starting centroids at \$2 and \$14.

- After the first assignment step, which customers belong to each cluster?
- After the first centroid update, where are the new centroids?
- Has the algorithm converged?
- Compute the within-cluster tightness (sum of squared distances from each customer to their centroid).

Answer

- Each customer goes to whichever centroid is closer:
 - \$2 is closer to \$2 $\rightarrow$ cluster A
 - \$4 is closer to \$2 (distance 2) than to \$14 (distance 10) $\rightarrow$ cluster A
 - \$6 is closer to \$2 (distance 4) than to \$14 (distance 8) $\rightarrow$ cluster A
 - \$14 $\rightarrow$ cluster B
 - \$16 $\rightarrow$ cluster B
 - \$18 $\rightarrow$ cluster B

Cluster A: {\$2, \$4, \$6}, Cluster B: {\$14, \$16, \$18}.
- New centroids are the average of each cluster's members:
 - Centroid A: $(2 + 4 + 6)/3 = \$4$
 - Centroid B: $(14 + 16 + 18)/3 = \$16$
- Yes.** If we ran another assignment step, no customer would switch clusters — so the algorithm has converged.
- Within-cluster tightness:
 - Cluster A: $(2 - 4)^2 + (4 - 4)^2 + (6 - 4)^2 = 4 + 0 + 4 = 8$
 - Cluster B: $(14 - 16)^2 + (16 - 16)^2 + (18 - 16)^2 = 4 + 0 + 4 = 8$
 - Total = **16**

8 Summary

What We Covered

Topic	Key Concepts
What is clustering?	Discovering natural groups in unlabeled data
A single cluster	Tight (members close together) and separated (far from others)
K-Means	Pick K, assign-and-update centroids until they stop moving
Elbow method	Plot tightness vs. K, look for the bend
Silhouette score	A number from -1 to $+1$ measuring cluster quality
Hierarchical clustering	Builds a dendrogram; pick K after the fact (use Ward linkage)
Customer segmentation	Cluster, profile, name, hand off to marketing